New Horizons®
Centro de Capacitación

Curso

Desarrollo Full Stack con C# y Angular

Docente: Edwin Barrientos



New Horizons®
Centro de Capacitación

Sesión 17 — Módulo 4:

Optimización y Performance Full Stack



Objetivo del módulo: optimizar el rendimiento y consumo de recursos de la solución Full Stack, aplicando técnicas de caching en memoria en C#, consultas LINQ de alta eficiencia y ChangeDetection OnPush con vistas diferidas @defer en Angular.



New Horizons®
Centro de Capacitación

Agenda

- Estrategias de Caching en .NET 8: IMemoryCache y preparación para IDistributedCache
- Optimización de consultas LINQ y EF Core: AsNoTracking, proyecciones y AsSplitQuery
- Estrategia ChangeDetectionStrategy.OnPush con Signals para renderizado ultrarrápido
- Vistas diferidas con bloques @defer, @placeholder y @loading en Angular 18
- Análisis y minimización del bundle con esbuild y verificación de presupuestos de peso



New Horizons®
Centro de Capacitación

Sesión 1:

Optimización y Rendimiento en Backend y Frontend

Visión general de la sesión

New Horizons®
Centro de Capacitación



Piensa en la cocina de un restaurante en hora punta. No gana el que cocina más rápido, sino el que no repite trabajo: ingredientes listos (caché), compras precisas (LINQ), meseros que solo atienden a quien llama (OnPush) y postres bajo pedido (@defer).

Backend que no repite

IMemoryCache y consultas LINQ/EF Core que traen solo lo necesario.

Piénsalo así: *La mise en place: lo que se usa siempre ya está picado y a la mano.*



Frontend que no se agota

OnPush con Signals y bloques @defer para pintar solo lo que cambia.

Piénsalo así: *El mesero que solo va a la mesa que toca el timbre.*



Entrega ligera

Análisis del bundle con esbuild y presupuestos de peso en angular.json.

Piénsalo así: *La maleta de mano: si pesa de más, no sube al avión.*

Pilares técnicos fundamentales

New Horizons
Centro de Capacitación

Cinco pilares, una sola idea: hacer menos trabajo, no trabajar más rápido.

1

Caching .NET 8

IMemoryCache hoy, IDistributedCache mañana.

La despensa junto a la cocina

2

LINQ + EF Core

AsNoTracking, proyecciones Select y AsSplitQuery.

La lista de compras precisa

3

OnPush + Signals

Re-render solo de componentes con cambios reales.

El timbre de cada mesa

4

@defer

@placeholder y @loading para vistas diferidas.

El postre bajo pedido

5

Bundle y budgets

esbuild + presupuestos de peso verificados.

La maleta de mano

Bloque 1: Registro de servicios de rendimiento

New Horizons
Centro de Capacitación

Program.cs

```
// Program.cs — servicios de rendimiento
1 builder.Services.AddMemoryCache();

2 // Preparado para escalar: misma idea, otro proveedor
// builder.Services.AddStackExchangeRedisCache(o =>
//     o.Configuration = builder.Configuration["Redis:Conn"]);

3 builder.Services.AddDbContext<AppDbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Db")));

4 builder.Services.AddResponseCompression(o =>
    o.EnableForHttps = true);

var app = builder.Build();
app.UseResponseCompression();
```



Equipar la cocina antes de abrir

Primero instalas la refrigeradora y los extractores; luego recién entran los pedidos.

1 AddMemoryCache()

La refrigeradora: memoria del propio servidor, sin infraestructura extra.

2 Redis, listo en la recámara

Cuando haya varias instancias, se cambia el proveedor, no la lógica.

3 AddDbContext

La cocina conectada a la despensa grande: SQL Server.

4 AddResponseCompression

Platos más livianos: JSON comprimido (gzip/brotli) viaja más rápido.

Práctica: Caching en memoria y LINQ optimizado

New Horizons
Centro de Capacitación

Receta de la práctica: configurar IMemoryCache, expiración deslizante y consultas LINQ optimizadas en el catálogo de centros de costo.



✅ **Cómo saber que funcionó:** la 1ª llamada a `/api/catalogos/centros-costo` va a SQL Server; la 2ª responde desde memoria y en el log de EF Core ya no aparece el SELECT. Termina con `dotnet build` sin advertencias.

IMemoryCache en acción: GetOrCreateAsync

New Horizons
Centro de Capacitación

CatalogoService.cs

```
1  public class CatalogoService(AppDbContext db, IMemoryCache cache)
   {
       public Task<List<CentroCostoDto>?> ObtenerCentrosCostoAsync() =>
2          cache.GetOrCreateAsync("catalogo:centros-costo",
               async entry =>
               {
3                  entry.SlidingExpiration = TimeSpan.FromMinutes(10);
                   entry.AbsoluteExpirationRelativeToNow =
                       TimeSpan.FromHours(1);

4                  return await db.CentrosCosto
                       .AsNoTracking()
                       .Select(c => new CentroCostoDto(c.Id, c.Nombre))
                       .ToListAsync();
               });
   }
```

La despensa junto a la cocina

Si el ingrediente ya está en la despensa, nadie sale al mercado (la base de datos).

Petición: centros de costo

¿Está en caché?

Sí (hit)
Responde desde memoria en microsegundos.

No (miss)
Va a SQL Server, guarda el resultado y responde.

Clave con prefijo (`catalogo:`): ordena y facilita invalidar.

Expiración e invalidación de la caché

New Horizons
Centro de Capacitación

CatalogoService.cs

```
public async Task ActualizarCentroCostoAsync(int id, string nombre)
{
    var cc = await db.CentrosCosto.FindAsync(id)
             ?? throw new KeyNotFoundException();

    cc.Nombre = nombre;
1   await db.SaveChangesAsync();

    // El dato cambió: la copia en memoria ya no sirve
2   cache.Remove("catalogo:centros-costo");
}
```

⏳ Deslizante (Sliding)

Se renueva cada vez que alguien la usa. Si nadie la pide en 10 min, se bota.

🗓️ Absoluta

Vence sí o sí a la hora, aunque se use mucho. Evita datos eternos.

🥛 La leche en la refrigeradora

Tiene fecha de vencimiento (absoluta) y, si nadie la toma en días, igual la desechas (deslizante).

1 Primero se guarda en la BD

La fuente de verdad siempre es SQL Server.

2 cache.Remove(clave)

Invalida al escribir: el siguiente GET vuelve a llenar la despensa.

🧠 “Solo hay dos cosas difíciles en informática: invalidar la caché y ponerle nombre a las cosas.”

IMemoryCache vs IDistributedCache (Redis)

New Horizons
Centro de Capacitación

📒 **Libreta personal vs pizarra de la oficina.** IMemoryCache es tu libreta: rapidísima, pero solo tú la ves. Redis es la pizarra compartida: todos los servidores leen lo mismo.

Criterio	📒 IMemoryCache	📄 IDistributedCache (Redis)
Dónde vive	RAM del propio proceso	Servidor externo (Redis)
Velocidad	Microsegundos, sin red	Milisegundos, viaja por red
Varias instancias	Cada una tiene su copia	Todas comparten el mismo dato
Tras reiniciar	Se pierde todo	Sobrevive al reinicio de la API
Úsalo cuando…	1 instancia, catálogos pequeños	Escalas horizontalmente o en contenedores

AsNoTracking: consultas de solo lectura

New Horizons
Centro de Capacitación

SolicitudQueries.cs

```
1  // Con tracking: EF guarda una copia de cada entidad
   var lista = await db.Solicitudes
       .Where(s => s.EstadoId == 1)
       .ToListAsync();

   // Solo lectura: sin snapshot en el ChangeTracker
   var lista = await db.Solicitudes
2      .AsNoTracking()
       .Where(s => s.EstadoId == 1)
       .ToListAsync();
```



Vitrina vs producto separado

Con tracking, el vendedor anota todo lo que miras por si lo compras. Si solo vas a mirar, que no anote nada.

Trabajo extra de EF Core (cualitativo)



1. **Para editar y guardar**

   Necesitas tracking: SaveChanges detecta los cambios.

2. **Para listar y reportar**

   AsNoTracking: menos memoria y menos CPU.

Proyecciones con Select: trae solo lo que pintas

New Horizons®
Centro de Capacitación

BandejaQuery.cs

```
var bandeja = await db.Solicitudes
    .AsNoTracking()
    .Where(s => s.EstadoId == EstadoSolicitud.Pendiente)
    .OrderByDescending(s => s.FechaCreacion)
1   .Select(s => new SolicitudResumenDto(
        s.Id,
        s.Motivo,
2       s.Solicitante.Nombre,
3       s.Detalles.Sum(d => d.Cantidad * d.PrecioUnitario)))
    .ToListAsync();
```



La lista de compras precisa

Para hacer un sándwich no traes todo el supermercado: solo pan, jamón y queso.

`SELECT *`

todas las columnas + entidades completas

`SELECT Id, Motivo, Nombre, SUM(...)`

solo 4 datos, calculados en SQL

1. **Select a DTO: sin tracking, contrato claro**
2. **Navegación sin Include: EF hace el JOIN**
3. **El total se suma en SQL, no en C#**

AsSplitQuery y la explosión cartesiana

New Horizons
Centro de Capacitación

DetalleQuery.cs

```
var solicitud = await db.Solicitudes
    .AsNoTracking()
1   .Include(s => s.Detalles)     // 20 filas hijas
    .Include(s => s.Historial)    // 15 filas hijas
2   .AsSplitQuery()               // 3 consultas separadas
    .FirstOrDefaultAsync(s => s.Id == id);
```

🚚 La mudanza en tres camiones

Un solo camión obliga a repetir cada mueble por cada caja. Tres camiones (uno por tipo) llevan cada cosa una vez.

💥 Una sola consulta (JOIN)

`1 × 20 × 15`

300 filas

✅ AsSplitQuery

`1 + 20 + 15`

36 filas

1 **Dos colecciones = producto cartesiano**

Cada detalle se combina con cada historial.

2 **AsSplitQuery**

Una consulta por colección; EF arma el objeto en memoria.

⚖️ **Trade-off:** más viajes a la BD y sin snapshot único entre consultas. Úsalo cuando incluyes 2 o más colecciones grandes.

El problema N+1 y cómo verlo en el log

New Horizons
Centro de Capacitación

NmasUno.cs

```
// N+1 (con lazy loading activo): una consulta por fila
foreach (var s in await db.Solicitudes.ToListAsync())
    total += s.Detalles.Count;   // dispara otra consulta

// Una sola consulta agregada en el servidor
var conteos = await db.Solicitudes
    .Select(s => new { s.Id, Items = s.Detalles.Count })
    .ToListAsync();

// Ver el SQL real en consola (solo en desarrollo)
options.LogTo(Console.WriteLine, LogLevel.Information)
       .EnableSensitiveDataLogging();
```

1

2

3



Un viaje por cada tornillo

Ir y volver por cada tornillo agota a cualquiera. Mejor llevar la lista completa en un solo viaje.

Bucle con lazy loading

1 + 100 = 101 consultas

Select agregado

1 consulta

3

LogTo en desarrollo

Si ves el mismo SELECT repetido en el log, tienes un N+1.

⚠ Nunca `EnableSensitiveDataLogging` en producción.

ChangeDetectionStrategy.OnPush con Signals

New Horizons®
Centro de Capacitación

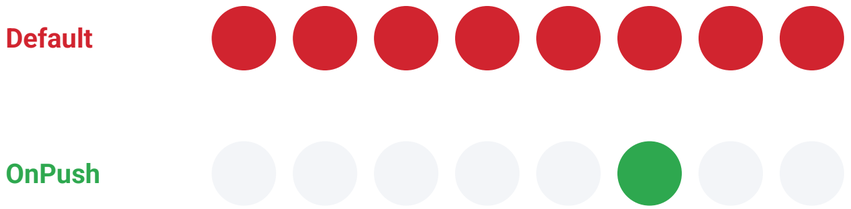
bandeja.component.ts

```
@Component({
  selector: 'app-bandeja',
  standalone: true,
1 changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h3>Pendientes: {{ total() }}</h3>
    @for (s of solicitudes(); track s.id) {
      <app-fila-solicitud [solicitud]="s" />
    }`
})
export class BandejaComponent {
  private api = inject(SolicitudService);
2 solicitudes = toSignal(this.api.pendientes(), { initialValue: [] });
3 total = computed(() => this.solicitudes().length);
}
```

El mesero y el timbre

Default: el mesero recorre TODAS las mesas cada vez. OnPush: solo va a la mesa que tocó el timbre.

Mesas revisadas en cada ciclo



1. **OnPush: se revisa solo con un “timbre”**
2. **toSignal: el Observable se vuelve signal**
3. **computed: se recalcula solo**

¿Qué despierta a un componente OnPush?

New Horizons®
Centro de Capacitación

timbres.ts

```ts
// Mutación: misma referencia, OnPush no se entera
this.items.push(nuevo);

// Nueva referencia: @Input detecta el cambio
this.items = [...this.items, nuevo];

// Con signals: update devuelve un arreglo nuevo
this.itemsSig.update(lista => [...lista, nuevo]);
```

La trampa clásica: el `push()` modifica el arreglo por dentro, pero la referencia sigue siendo la misma. Para OnPush es como si nadie hubiera tocado el timbre.

Los 4 timbres

1. Cambia la referencia de un @Input
2. Evento del template: (click), (input)…
3. Una signal leída en el template cambia
4. El pipe async emite o markForCheck()

Vistas diferidas con @defer

New Horizons
Centro de Capacitación

dashboard.component.html

```
1 @defer (on viewport) {
    <app-grafico-gastos [datos]="resumen()" />
2 } @placeholder (minimum 500ms) {
    <div class="skeleton-grafico"></div>
3 } @loading (after 100ms; minimum 1s) {
    <mat-spinner diameter="32" />
4 } @error {
    <p>No se pudo cargar el gráfico.</p>
  }
```



El postre bajo pedido

No se prepara al abrir el restaurante: se sirve cuando el cliente llega a esa parte de la comida.

1. **@defer (on viewport)**
   Descarga el componente al hacerse visible.

2. **@placeholder**
   El plato vacío: lo que se ve antes de pedir.

3. **@loading**
   “Ya sale”: spinner mientras descarga.

4. **@error**
   Plan B si falla la descarga del chunk.

Triggers de @defer y diferencia con lazy routes

New Horizons
Centro de Capacitación

Los sensores de la puerta automática: tú eliges qué evento abre la puerta y descarga el bloque.

`on idle`

Cuando el navegador queda libre (por defecto).

`on viewport`

Cuando el bloque entra en pantalla.

`on interaction`

Al hacer clic o escribir en un elemento.

`on hover`

Cuando el mouse pasa por encima.

`on timer(2s)`

Después de un tiempo fijo.

`when cond()`

Cuando una condición pasa a true.

Lazy routes

Divide por página completa (loadComponent en la ruta).

@defer

Divide dentro de la página, bloque por bloque, con prefetch on idle.

Análisis del bundle con esbuild

New Horizons
Centro de Capacitación

terminal

```
# 1. Build de producción con estadísticas
ng build --configuration production --stats-json

# 2. Se genera el metafile de esbuild
#    dist/solicitudes-app/stats.json

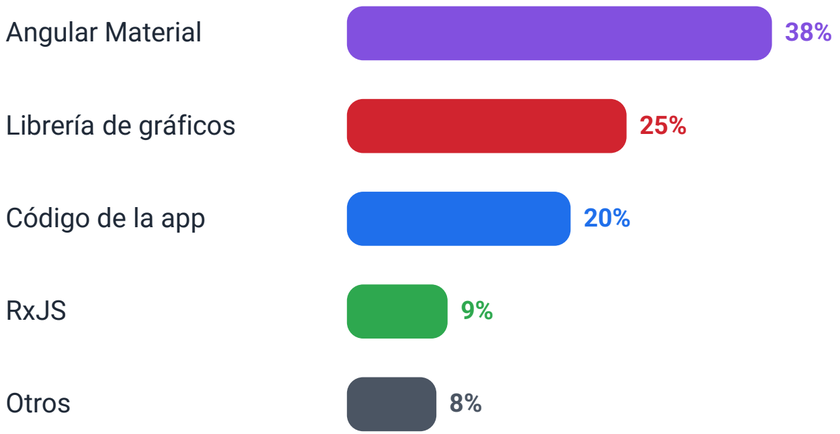
# 3. Arrastra stats.json a:
#    https://esbuild.github.io/analyze/

# 4. Revisa la salida de la consola:
#    Initial total  /  Lazy chunk files
```

La radiografía de la maleta

Antes de culpar a la aerolínea, mira qué llevas dentro y qué puedes dejar en casa.

Ejemplo ilustrativo de main.js



Lo pesado y secundario → `@defer` o lazy route.

Presupuestos de peso (budgets) en angular.json

New Horizons
Centro de Capacitación

angular.json

```
"configurations": {
  "production": {
    "budgets": [
      {
1       "type": "initial",
2       "maximumWarning": "500kB",
3       "maximumError": "1MB"
      },
      {
4       "type": "anyComponentStyle",
        "maximumWarning": "4kB",
        "maximumError": "8kB"
      }
    ]
  }
}
```

La balanza de la aerolínea

A los 500 kB te avisan; pasado 1 MB, la maleta no sube: el build falla.

Bundle inicial



1. **initial: lo que baja al abrir la app**
2. **maximumWarning: aviso amarillo**
3. **maximumError: el build falla**
4. **anyComponentStyle: CSS por componente**

Errores comunes y cómo evitarlos

New Horizons®
Centro de Capacitación

✗ Evita esto	✓ Haz esto
Omitir la validación estricta de parámetros en el servidor	Validar siempre los contratos DTO en cada llamada HTTP
Almacenar información sensible en repositorios públicos	Utilizar variables de entorno seguras en desarrollo y producción
Desacoplar la UI de las políticas de seguridad del backend	Mantener arquitectura standalone y tipado estricto
Cachear sin estrategia de invalidación	Expiración absoluta + cache.Remove() al actualizar
Mutar arreglos (push) en componentes OnPush	Crear nuevas referencias o usar signal.update()

Checklist de performance antes de producción

New Horizons®
Centro de Capacitación



La parada en pits de Fórmula 1: segundos de revisión que evitan perder la carrera en producción.

✓	**Catálogos frecuentes en IMemoryCache con expiración absoluta.**	→ *Menos viajes a SQL Server.*
✓	**Todo GET de listados con AsNoTracking + Select a DTO.**	→ *Menos memoria por request.*
✓	**AsSplitQuery cuando incluyes 2+ colecciones.**	→ *Sin explosión cartesiana.*
✓	**Componentes de presentación con OnPush + Signals.**	→ *Menos ciclos de detección.*
✓	**Bloques pesados bajo @defer con @placeholder.**	→ *Carga inicial más corta.*
✓	**Budgets en angular.json revisados en el pipeline.**	→ *Ningún bundle obeso llega a prod.*

Resumen de la Sesión 17

New Horizons®
Centro de Capacitación

Caching en memoria

El caching en memoria reduce la latencia de consultas repetitivas de catálogos a microsegundos.

≈ La despensa

Consultas eficientes

AsNoTracking y proyecciones Select alivian el ChangeTracker y reducen la memoria del servidor.

≈ La lista de compras

OnPush + Signals

OnPush junto a Signals garantiza que solo se re-rendericen componentes con cambios reales.

≈ El timbre de la mesa

Vistas diferidas

Los bloques @defer postergan la descarga de componentes secundarios hasta que sean visibles.

≈ El postre bajo pedido

Respuesta inmediata

La solución alcanza tiempos de respuesta inmediatos y alta concurrencia con bajo consumo.

≈ La cocina en hora punta

Listo para producción

Solución enterprise lista para producción, con budgets que vigilan el peso.

≈ La maleta de mano

PREGUNTAS...

New Horizons®
Centro de Capacitación

1. ¿Cuándo conviene utilizar IMemoryCache y cuándo una caché distribuida como Redis?
2. ¿Por qué ChangeDetectionStrategy.OnPush reduce drásticamente el consumo de CPU en Angular?
3. ¿Cómo evita AsSplitQuery la explosión cartesiana en consultas con múltiples tablas hijas?
4. ¿Qué ventaja aporta @defer frente al lazy loading tradicional de rutas?
5. ¿Cómo se configuran los presupuestos (budgets) en angular.json para alertar sobre bundles pesados?

New Horizons®
Centro de Capacitación

Conclusiones

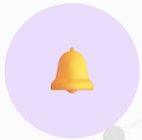
Despensa a la mano

El caching en memoria reduce la latencia de consultas repetitivas de catálogos a microsegundos.



Compras precisas

AsNoTracking y proyecciones Select alivian el ChangeTracker y reducen la memoria del servidor.



Solo quien toca el timbre

OnPush junto a Signals garantiza que solo se re-rendericen componentes con cambios reales.



Postre bajo pedido

Los bloques @defer postergan la descarga de componentes secundarios hasta que sean visibles.



Rápido y liviano

La solución alcanza tiempos de respuesta inmediatos y alta concurrencia con bajo consumo.

GRACIAS